

Programação Funcional

Aula 06 - Cálculo Lambda e Funções Anônimas

Eliseu C. Miguel

Universidade Federal de Alfenas

11 de agosto de 2025

Tópicos

- 1 Cálculo Lambda e Funções Anônimas
- 2 Uso com funções de Alta Ordem
- 3 Múltiplos Argumentos e Casamento de Padrões
- 4 Exercícios
- 5 Referências

Tópicos

- 1 Cálculo Lambda e Funções Anônimas
- 2 Uso com funções de Alta Ordem
- 3 Múltiplos Argumentos e Casamento de Padrões
- 4 Exercícios
- 5 Referências

Tópicos

- 1 Cálculo Lambda e Funções Anônimas
- 2 Uso com funções de Alta Ordem
- 3 Múltiplos Argumentos e Casamento de Padrões
- 4 Exercícios
- 5 Referências

Tópicos

- 1 Cálculo Lambda e Funções Anônimas
- 2 Uso com funções de Alta Ordem
- 3 Múltiplos Argumentos e Casamento de Padrões
- 4 Exercícios
- 5 Referências

Tópicos

- 1 Cálculo Lambda e Funções Anônimas
- 2 Uso com funções de Alta Ordem
- 3 Múltiplos Argumentos e Casamento de Padrões
- 4 Exercícios
- 5 Referências

Funções Anônimas (Lambdas)

- Até agora, todas as funções que criamos tinham um nome (ex: soma, double).
- Muitas vezes, precisamos de uma função pequena, usada apenas uma vez, como argumento para uma função de alta ordem (ex: map, filter).
- Criar um nome para uma função de uso único pode poluir o código.
- **Funções anônimas**, ou **expressões lambda**, nos permitem criar funções "em tempo real", sem a necessidade de um nome.

Sintaxe

```
\parametro -> expressao
```

Exemplos de Sintaxe

- Uma expressão lambda é um valor, assim como 5 ou True. O valor, neste caso, é uma função.

Função com Nome

```
somaUm :: Int -> Int  
somaUm x = x + 1
```

Função Anônima Equivalente

```
\x -> x + 1
```

Testando no GHCi/Hugs

```
ghci> (\x -> x + 1) 5  
6
```

Uso com Funções de alta ordem

- O principal uso de lambdas é como argumento para funções como `map` e `filter`, tornando o código mais conciso.

Sem lambda

```
f x = 2*x + 1  
map f [1,2,3]
```

Com lambda

```
map (\x -> 2*x + 1) [1,2,3]  
= [3, 5, 7]
```

Uso com Funções de alta ordem

Exemplo com filter

```
filter (\s -> length s > 5) ["ola", "haskell", "top"]  
= ["haskell"]
```

Múltiplos Argumentos

- Lambdas também podem receber múltiplos argumentos e usar casamento de padrão simples (os argumentos são listados antes da seta).

Múltiplos Argumentos

```
\x y -> x + y  
ghci> (\x y -> x + y) 3 4  
7
```

Casamento de Padrão

- Podemos usar padrões, como tuplas, diretamente na declaração da lambda.

Casamento de Padrão

```
map (\( x,y) -> x + y) [(1,2), (3,4)]  
= [3, 7]  
map (\(x:xs) -> x)    [[1,2], [3,4]]  
= [1, 3]
```

Exercícios

- 1 Use `map` e uma expressão lambda para converter uma lista de strings em uma lista com os seus respectivos tamanhos. Ex: `["a", "b", "c"] → [1, 2, 3]`.
- 2 Use `filter` e uma lambda para encontrar todos os números em uma lista `[1..50]` que são múltiplos de 3 e 5 ao mesmo tempo.
- 3 Crie uma função que recebe uma lista de tuplas `[(String, Int)]` e usa `filter` com uma lambda para reter apenas os pares cuja string tem mais de 5 caracteres.

Referências Bibliográficas

Slides [1] Livro [2]



Vladimir Oliveira Di Iorio.

Programação Funcional - Tipos Básicos e Programas Simples em Haskell.

DPI - UFV.



Miran Lipovaca.

Learn You a Haskell for Great Good!

Lipovaca.

Sobre as apresentações didáticas

Atenção: este material não é completo para seus estudos.
Ao contrário, é apenas um guia para lhe informar quais são os
tópicos importantes a serem estudados.

O estudo completo e necessário dá-se por materiais específicos
sobre cada tópico, como livros e conteúdo na Internet.

Selecione materiais de fontes de seu interesse considerando a
bibliografia citada no plano de ensino da disciplina para estudar os
tópicos aqui abordados.

Consulte o professor, em caso de dúvidas

Agradecimentos especiais

Agradeço ao monitor da disciplina Felipe Araújo Correia por ter elaborado esta apresentação.

Agradeço a toda a comunidade $\text{\LaTeX}$.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.